

Fixing Lazy Sender Algorithm Customization

Document #: P3303R1
Date: 2024-06-24
Project: Programming Language C++
Audience: LEWG Library Evolution
Reply-to: Eric Niebler
<eric.niebler@gmail.com>

Contents

- [1 Introduction](#)
- [2 Executive Summary](#)
- [3 Revision History](#)
- [4 Discussion](#)
- [5 Proposed Wording](#)
- [6 References](#)

“Experience is simply the name we give our mistakes.”

— Oscar Wilde

“To err is human, but to really foul things up you need a computer.”

— Paul R. Ehrlich

§ 1 Introduction

The paper [[P2999R3](#)] “Sender Algorithm Customization” proposed, among other things, to make sender algorithms customizable lazily; that is, when their senders are connected with receivers. LEWG agreed and forwarded P2999 to LWG. Due to a gross oversight, however, P2999 didn’t propose the wording changes to connect and `get_completion_signatures` that actually implement the design that LEWG approved. This paper corrects the oversight by using the new `transform_sender` utility in `connect` and `get_completion_signatures` as P2999 promised and failed to do.

§ 2 Executive Summary

The changes this paper proposes are:

1. `get_completion_signatures(sndr, env)` first transforms `sndr` with `transform_sender(get-domain-late(sndr, env), sndr, env)` and then uses the result in place of `sndr`.
2. `connect(sndr, rcvr)` first transforms `sndr` with `transform_sender(get-domain-late(sndr, get_env(rcvr)), sndr, get_env(rcvr))` and then uses the result in place of `sndr`.

§ 3 Revision History

R1

- Fix specification of `connect` and `get_completion_signatures` to not cause the evaluation of the `sndr` expression when computing the type of the domain to be passed to `transform_sender`.

R0

- Initial revision

§ 4 Discussion

Table 2 in [P2999R3] shows how that paper proposed to change the `connect` customization point. The table is reproduced below. Note that P2999R3 was targeting a version of P2300 that still employed `tag_invoke`, which has since been removed.

Table 2 from P2999R3: The addition of `transform_sender` to `connect`

Before	After
<pre>struct connect_t { template <receiver Receiver, sender_in<env_of_t<Receiver>> Sender> requires /* ... */ auto operator()(Sender&& snd, Receiver&& rcv) const { // First, look for a customization of // tag_invoke: if constexpr (tag_invocable<connect_t, Sender, Receiver>) { return tag_invoke(*this, std::forward<Sender>(snd), std::forward<Receiver>(rcv)); } // Next, see if the sender is // co_await-able:</pre>	<pre>struct connect_t { template <receiver Receiver, sender_in<env_of_t<Receiver>> Sender> requires /* ... */ auto operator()(Sender&& snd, Receiver&& rcv) const { // Apply any sender transformations // using the receiver's domain: auto&& snd2 = transform_sender(get- domain-late(snd, get_env(rcv)), std::forward<Sender>(snd), get_env(rcv)); using Sender2 = decltype(snd2); // First, look for a customization of // tag_invoke: if constexpr (tag_invocable<connect_t, Sender2, Receiver>) { return tag_invoke(*this, std::forward<Sender2>(snd2), std::forward<Receiver>(rcv));</pre>

Before	After
<pre> else if constexpr (is-await- connectable<Sender, Receiver>) { /* ... */ } }; </pre>	<pre> } // Next, see if the sender is // co-await-able: else if constexpr (is-await- connectable<Sender2, Receiver>) { /* ... */ } }; </pre>

The design shown above is the one that was discussed and voted on in LEWG.

A glance at the [proposed wording](#) from P2999R3 shows that no such change was ever made. This *most critical* part of that paper’s design intent was inadvertently left out of the wording. Face, meet palm.

This paper proposes no design changes from those described in P2999R3. It “merely” corrects the wording to agree with the design.

§ 5 Proposed Wording

[Editor's note: The changes in this paper are relative to [P2300R9].]

[Editor's note: Change [exec.getcomplsigs] as follows:]

execution::get_completion_signatures [exec.getcomplsigs]

1. `get_completion_signatures` is a customization point object. Let `sndr` be an expression such that `decltype((sndr))` is `Sndr`, and let `env` be an expression such that `decltype((env))` is `Env`. Let new sndr be the expression `transform_sender(decltype(get-domain-late(sndr, env))){}, sndr, env`, and let `NewSndr` be `decltype((new sndr))`. Then `get_completion_signatures(sndr, env)` is expression-equivalent to:

1. `decltype(new sndr.get_completion_signatures(env)){} if that expression is well-formed,`
2. Otherwise,
`remove_cvref_t<NewSndr>::completion_signatures{} if that expression is well-formed,`
3. Otherwise, if `is-awaitable<NewSndr, env-promise<Env>>` is true, then:

```

completion_signatures<
    SET-VALUE-SIG(await-result-type<NewSndr, env-
        promise<Env>>), // see [exec.snd.concepts]
    set_error_t(exception_ptr),
    set_stopped_t()>{}

```

4. Otherwise, `get_completion_signatures(sndr, env)` is ill-formed.
2. Let `rcvr` be an rvalue receiver of type `Rcvr`, and let `Sndr` be the type of a sender such that `sender_in<Sndr, env_of_t<Rcvr>>` is true. Let `Sigs...` be the template arguments of the `completion_signatures` specialization named by `completion_signatures_of_t<Sndr, env_of_t<Rcvr>>`. Let `CSO` be a completion function. If sender `Sndr` or its operation state cause the expression `CSO(rcvr, args...)` to be potentially evaluated ([basic.def.odr]) then there shall be a signature `Sig` in `Sigs...` such that `MATCHING-SIG(decayed-typeof<CSO>(decltype(args)...), Sig)` is true ([exec.general]).

[Editor's note: Change [exec.connect] paragraphs 2 and 6 as shown below. Paragraphs 3-5 are unchanged but are shown here in their entirety to give context to the surrounding changes.]

execution::connect [exec.connect]

1. `connect` connects ([async.ops]) a sender with a receiver.
2. The name `connect` denotes a customization point object. For subexpressions `sndr` and `rcvr`, let `Sndr` be `decltype((sndr))` and `Rcvr` be `decltype((rcvr))`, let new sndr be the expression `transform_sender(decltype(get_domain_late(sndr, get_env(rcvr))) {}, sndr, get_env(rcvr))`, let `NewSndr` be `decltype((new sndr))`, and let `DS` and `DR` be the decayed types of `NewSndr` and `Rcvr`, respectively.
3. Let `connect-awaitable-promise` be the following class:

```
namespace std::execution {
    struct connect-awaitable-promise
        : with-await-transform<connect-awaitable-promise> {
        DR& rcvr; // exposition only

        connect-awaitable-promise(DS&, DR& rcvr) noexcept :
            rcvr(rcvr) {}

        suspend_always initial_suspend() noexcept { return {}; }
        [[noreturn]] suspend_always final_suspend() noexcept {
            terminate(); }
        [[noreturn]] void unhandled_exception() noexcept {
            terminate(); }
        [[noreturn]] void return_void() noexcept { terminate(); }

        coroutine_handle<> unhandled_stopped() noexcept {
            set_stopped((DR&&) rcvr);
            return noop_coroutine(); }
    };
}
```

```

    }

    operation-state-task get_return_object() noexcept {
        return operation-state-task{
            coroutine_handle<connect-awaitable-
                promise>::from_promise(*this)};
    }

    env_of_t<const DR&> get_env() const noexcept {
        return execution::get_env(rcvr);
    }
};
}

```

4. Let *operation-state-task* be the following class:

```

namespace std::execution {
    struct operation-state-task {
        using operation_state_concept = operation_state_t;
        using promise_type = connect-awaitable-promise;
        coroutine_handle<> coro; // exposition only

        explicit operation-state-task(coroutine_handle<> h)
            noexcept : coro(h) {}
        operation-state-task(operation-state-task&& o) noexcept
            : coro(exchange(o.coro, {})) {}
        ~operation-state-task() { if (coro) coro.destroy(); }

        void start() & noexcept {
            coro.resume();
        }
    };
}

```

5. Let *V* name the type *await-result-type*<*DS*, *connect-awaitable-promise*>, let *Sigs* name the type:

```

completion_signatures<
    SET-VALUE-SIG(V), // see [exec.snd.concepts]
    set_error_t(exception_ptr),
    set_stopped_t()>

```

and let *connect-awaitable* be an exposition-only coroutine defined as follows:

```

namespace std::execution {
    template<class Fun, class... Ts>
    auto suspend-complete(Fun fun, Ts&&... as) noexcept { //
        exposition only
        auto fn = [&, fun]() noexcept { fun(std::forward<Ts>
            (as)...); };

    struct awaiter {
        decltype(fn) fn;
    };
}

```

```

    static constexpr bool await_ready() noexcept { return
        false; }
    void await_suspend(coroutine_handle<>) noexcept {
        fn(); }
    [[noreturn]] void await_resume() noexcept {
        unreachable(); }
};
return awaiter{fn};
};

operation-state-task connect-awaitable(DS sndr, DR rcvr)
    requires receiver_of<DR, Sigs> {
    exception_ptr ep;
    try {
        if constexpr (same_as<V, void>) {
            co_await std::move(sndr);
            co_await suspend-complete(set_value,
                std::move(rcvr));
        } else {
            co_await suspend-complete(set_value,
                std::move(rcvr), co_await std::move(sndr));
        }
    } catch(...) {
        ep = current_exception();
    }
    co_await suspend-complete(set_error, std::move(rcvr),
        std::move(ep));
}
}

```

6. If `Sndr` does not satisfy sender or if `Rcvr` does not satisfy receiver, `connect(sndr, rcvr)` is ill-formed. Otherwise, the expression `connect(sndr, rcvr)` is expression-equivalent to:

1. `new_sndr.connect(rcvr)` if that expression is well-formed.
 — *Mandates:* The type of the expression above satisfies `operation_state`.
2. Otherwise, `connect-awaitable(new_sndr, rcvr)` if that expression is well-formed.
3. Otherwise, `connect(sndr, rcvr)` is ill-formed.

§ 6 References

[P2300R9] Eric Niebler, Michał Dominiak, Georgy Evtushenko, Lewis Baker, Lucian Radu Teodorescu, Lee Howes, Kirk Shoop, Michael Garland, Bryce Adelstein Lelbach. 2024-04-02. ``std::execution``.

<https://wg21.link/p2300r9>

[P2999R3] Eric Niebler. 2023-12-13. Sender Algorithm Customization.

<https://wg21.link/p2999r3>